

# Intelligentni agenti

Agent je bilo šta što može da precipira svoje okruženje preko senzora i deluje na okruženje preko efektora.

Može se opisati pomoći funkcije koja preslikava opažaje u akcije. Izvršava se na fizičkoj arhitekturi kako bi realizovao te funkcije.

Intelligentni agent je računarski sistem koji može da izvrši autonomno fleksibilnu akciju u nekom okruženju za dostizanje definisanog cilja. Pod fleksibilnom akcijom podrazumevamo:

1. Reaktivnost – postoji interakcija sa okruženjem i sposoban je da odgovori na promene koje se dešavaju u njemu
2. Pro-aktivnost – sposoban je za ponašanje definisano ciljem
3. Socijalni aspekt – sposoban je da interaguje sa drugim agentima (i ljudima)

## Klasifikacija agenata

---

### po tipu

1. Hardverski – roboti koji imaju svoje senzore za istraživanje okruženja i efektore za delovanje. (podrazumeva se da imaju i neki SW)
2. Softverski – aplikacije čije su okruženje druge aplikacije.

### po mobilnosti

1. Statični – permanentno locirani na jednoj lokaciji
2. Mobilni – mogu da menjaju svoju lokaciju. Kod SW se odnosi na njihovu sposobnost da se izvršavaju na nekom računaru u lokalnoj mreži ili na Internetu.

### po arhitekturi

Arhitektura agenta definiše da li agent

1. ima samo osnovne osobine reakcije na nadražaje iz okoline na osnovu unapred definisanih informacija o mogućim reakcijama
2. ili ima složenije funkcije planiranja i predviđanja budućih akcija.

### veličina i inteligencija

Za SW su međusobno povezane. Inteligencija je uslovljena znanjem i zavisi od sposobnosti agenta da uči. Što je agent veći, postoji veća mogućnost za ugrađivanje znanja i algoritama za učenje.

## Evolucija agenta

---

Racionalnost: da li se agent ponaša tako da maksimizira performanse na osnovu zadatog znanja i opažanja okruženja

Autonomnost: da li su agentove akcije određene iskustvom

Idelani racionalni agent za svaku sekvencu opažanja maksimizira performanse, na osnovu ulaza i ugrađenog znanja.

Ovakvo ponašanje je limitirano HW mogućnostima.

### Merenje performansi

1. Eksterno merenje performansi – kako agent deluje na okolinu
2. Samo-evaluacija - agent je sposoban da sam vrši procenu svojih akcija

kontinualno, periodično ili jedna evaluacija

## Osnovni koncepti

---

1. Okruženje
2. Opažanja / senzori

3. Akcije / efektori
4. Mera performansi
5. Cilj

## Vrste agenata – arhitektura

### Table-driven agenti

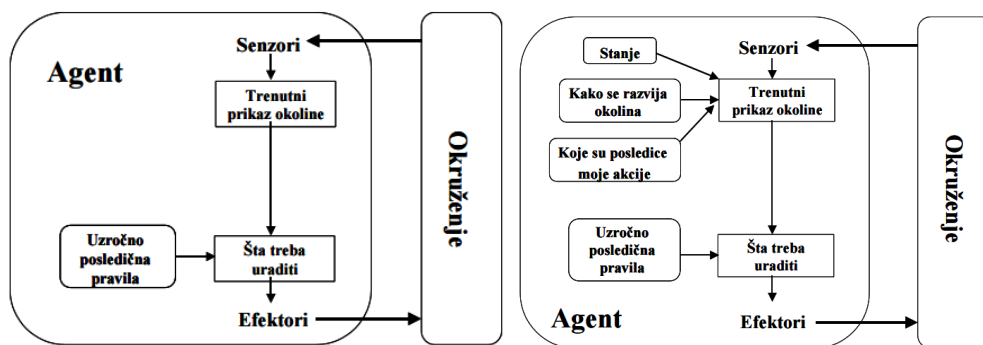
Funkcija preslikavanja je tabela koja sadrži sva moguća preslikavanja. Problem je što može biti prevelika.

Algoritam: ažuriraj memoriju (nova opažanja), izaberi najbolju akciju, ažuriraj memoriju (nova akcija)

### Refleks agenti

Deluju samo na osnovu trenutne percepcije. Funkcija agenta uspeva samo ukoliko je okolina u potpunosti primetljiva. Moguće su beskonačne petlje.

Algoritam: interpretacija ulaza, izbor pravila (poklapanje sa situacijom), nova akcija na osnovu pravila



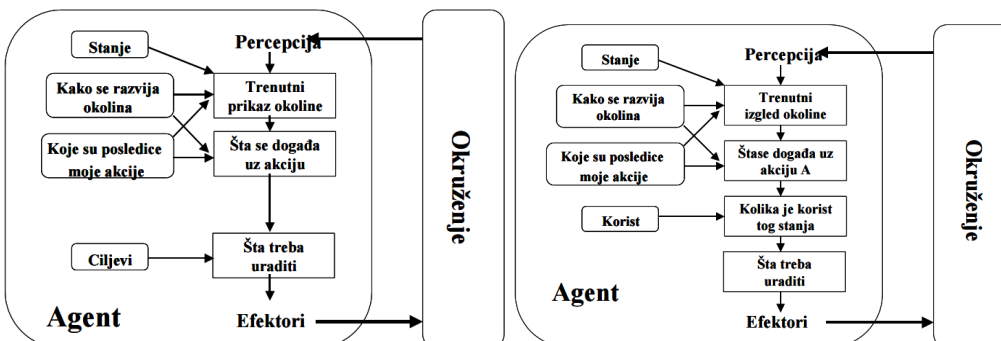
### Refleks agent koji pamti stanje (bazirani na modelu)

Oni mogu raditi i sa delimično primetljivom okolinom. On čuva trenutno stanje u nekoj strukturi koja opisuje deo sveta koji ne može biti viđen. To znanje kako se razvija okolina je nazvano model sveta. Ovi agenti održavaju neku vrstu internog modela koji zavisi od istorije percepcije i tako održava bar deo neprimetnog aspekta trenutnog stanja. Bira akciju na isti način kao i jednostavni refleksi agent.

Algoritam: ažuriraj stanje na osnovu ulaza, poklapanje sa pravilima, izbor akcije na osnovu pravila, ažuriraj stanje na osnovu akcije

### Agent baziran na cilju

On proširuje mogućnosti agenta koji pamti stanje korišćenjem informacije o cilju koja opisuje situaciju koja je poželjna. Tako agent bira ona akciju koja ispunjava ciljano stanje.



### Agent baziran na korisnosti

Agenti bazirani na cilju samo znaju razliku između ciljnog i ne-ciljnog stanja. Moguće je definisati meru kojom se meri poželjnost stanja. Ona se može dobiti upotrebom funkcije korisnosti. Agent bira akciju koja maksimizira očekivanu korisnost posledice akcije.

## Tipovi okruženja

1. Potpuno ili delimično dostupna – da li agent ima podatke o celukupnom okruženju ili samo nekom delu okruženja
2. Deterministička ili stohastička – da li sledeće stanje klompletno zavisi od tekućeg i izabrane akcije
3. Diskretna ili kontinualna – da li postoji konačan broj jasno definisanih koncepata i akcija
4. Bazirana na epizodama ili sekvencijalna – da li akcije zavise samo od jedne epizode ili su one međusobno zavisne
5. Statička ili dinamička – da li se okruženje ne menja dok agent razmišlja
6. Konkurentna ili komparativna – da li agenti međusobno sarađuju ili se nadmeću

## CSP

### Definicija

- Konačan skup promenljivih  $V_1, V_2, \dots, V_n$ , neprazni domeni mogućih vrednosti za svaku promenljivu  $D_{v1}, D_{v2}, \dots, D_{vn}$
- Konačan skup ograničenja  $C_1, C_2, \dots, C_m$
- Stanje se definiše kao dodela vrednosti nekim promenljivama. Konzistentna dodela je ona koja ne narušava ograničenja.
- Rešenje za CSP je kompletna dodela (uključene su sve promenljive) koja zadovoljava sva ograničenja.

Primeri: Scheduling, kriptografija, computer vision, bojenje mape, grafa

### Vrste CSP

1. Diskretne promenljive
  - a. Konačni domeni –  $d^n$  rešenja, pr. boolean CSP
  - b. Beskonačni domeni – beskonačno mnogo rešenja, pr. job scheduling, linearna ograničenja, nelinearna (nema generalnog algoritma)
2. Kontinualne promenljive – Određivanje raspodera letenja ili časova, linearna ograničenja

### Vrste ograničenja

1. Unarno ograničenje – utiče na samo jednu promenljivu
2. Binarno - utiče na par
3. Ograničenje višeg reda – 3 ili više, uvek može biti predstavljeno preko binarnih ograničenja
4. Preferenca (soft ograničenja)

## CSP se može predstaviti kao standardni problem traženja (po dubini).

- Početno stanje: prazna dodela  $\{\}$
- Funkcija sledbenika: dodela vrednosti bilo kojoj nedodeljenoj promenljivoj tako da ne narušava ograničenje
- Test na ciljno stanje: tekuća dodela je kompletna
- Cena puta: konstantna cena za svaki korak (nija važna, nema uticaj na brzinu traženja)

CSP su komutativni: redosled dodele nije važan

Svi CSP algoritmi traženja mogu da generišu sledbenike razmatranjem dodele samo za jednu promenljivu u svakom čvoru stabla traženja!

### Backtracking traženje

Bira vrednosti za jednu promenljivu i vraća se kada ta promenljiva nema legalnih vrednosti za dodelu. Neinformranisani algoritam – generalno nema dobre performanse.

# Povećanje efikasnosti CSP

---

## Minimum remaining values (MRV) heuristika

Bira promenljivu koja ima najmanje legalnih vrednosti. Cilj je izabrati promenljivu koja će prouzrokovati grešku što ranije, omogućavajući odsecanje kod stabla što ranije.

## Degree heuristic (DH)

Bira promenljivu sa najviše ograničenja (najviše potega u grafu).

## Least constraining value heuristic (LCV)

Bira promenljivu koja najmanje ograničavajuću vrednost tj. onu koja najmanje smanjuje moguće vrednosti preostalih promenljivih. Ostavlja maksimalnu fleksibilnost za rešenje.

MRV i DH biraju promenljivu koja najverovatnije dovodi do neuspeha a LCV pokušava da izbegne neuspeh.

## Forward checking

Prate se preostale legalne vrednosti za nedodjeljene promenljive koje su povezane sa tekućom i prekida se traženje kada bilo neka nema legalne vrednosti za dodelu.

Propagacija ograničenja propagira konzistentnost potega kroz graf. CP i FC eliminišu deo prostora traženja.

## Arc-consistency (AC)

Sistemska procedura za propagaciju ograničenja. Poteg  $X \rightarrow Y$  je konzistentan akko za svaku vrednost  $x$  iz  $X$  postoji neko dozvoljeno  $y$  iz  $Y$ .

Ako  $X$  gubi vrednost, susede od  $X$  treba ponovo proveriti jer dolazni potezi mogu postati nekonzistentni ponovo. Svaki put kada se domen promenljive menja, sve dolazne poruke se moraju ponovo slati. Ponavljati sve dok nema pouka koje menjaju bilo koji domen. FC je prvi korak iz AC.

# Genetski algoritmi

U svakoj ćeliji se nalazi skup hromozoma, oni sadže DNK lance koji opisuju ceo organizam. Hromozomi se sastoje iz gena, svaki gen opisuje pojedinačni protein odnosno pojedinačnu osobinu. U toku reprodukcije dolazi do rekombinovanja gena roditelja čime se formiraju novi hromozomi. Novi potomak može da mutira što dovodi do malih promena DNK elemenata. Dobrota novonastalog organizma se menja njegovim uspehom u životu (preživljavanju).

Prostor traženja je skup rešenja u kome se traži najbolje rešenje. Svaka tačka u prostoru traženja je jedno rešenje (koje ima svoju dobrotu (fitness)). Traženje rešenja je ekvivalentno traženju ekstremnih vrednosti u prostoru traženja. Nekada je prostor traženja jasno definisan ali su poznate samo određene tačke. Korišćenjem genetskih algoritama, proces traženja generiše druge tačke (moguća rešenja).

Genetski algoritmi se primenjuju kada se problem ne može rešiti algoritamski, ali je moguće proveriti valjanost pojedinih rešenja. GA ne garantuju najbolje rešenje već zadovoljavajuće.

Algoritam zahteva polazni skup rešenja (populacija). Ona se koristi da formiraju novu populaciju u nadi da će ona biti bolja (izbor rešenja za novu populaciju bira se na osnovu dobrote). Ovaj proces se ponavlja se dok se ne zadovolji određeni uslov (broj populacija, poboljšanje najboljeg rešenja).

Algoritam:

1. [početak] Generiše se slučajna populacija od  $n$  hromozoma.
2. [dobrota] Računa se dobrota  $f(x)$  za svaki hromozom iz populacije.
3. [nova populacija] se kreira ponavljanjem sledećih koraka
  - a. [izbor roditelja] Biraju se dva hromozoma iz tekuće populacije prema dobroti
  - b. [rekombinacija] Novi hromozom se formira kombinovanjem dva izabrana hromozoma
  - c. [mutacija] novi hromozom mutira sa određenom verovatnoćom
  - d. [dodavanje potomaka] novi hromozom se dodaje u novu populaciju
4. [zamena populacija] novogenerisana populacija postaje tekuća populacija
5. [uslov za kraj] ukoliko je zadovoljen uslov, vraća se najbolje rešenje iz tekuće populacije

6. [novi ciklus] ako nije ispunjen uslov, algoritam se ponavlja od 2. koraka

## Kodiranje hromozoma

---

### Binarno kodiranje

Predstavljanje hromozoma nizom bitova. Najčešće se koristi zbog jednostavnosti ali nije pogodan za određene primene pa su često neophodne korekcije nakon rekombinacije i/ili mutacije. Primer problem ranca. Pr. (oblačno ili kišovito) i (jak vetar) 011 10.

#### rekombinacija

Kopiraju se određeni bitovi jednog odnosno drugog roditelja. Bit na poziciji  $i$  je kopiran sa bita sa pozicije  $i$  jednog od roditelja. Izbor od kog roditelja se određuje na osnovu maske rekombinacije.

1. sa jednom tačkom prelazka – maska počinje nizom od  $n$  uzastopnih jedinica i potrebnim brojem nula da bi se kompletirao niz. Svaki put se broj  $n$  bira na slučajan način.
2. sa dve tačke prelazka – počinje sa  $n_0$  nula, zatim  $n_1$  jedinica i neophodan broj nula da bi se kompletirao niz. Rand.
3. uniformno ukrštanje – maska je slučajan niz nula i jedinica.

#### mutacija

Formira potomka samo od jednog roditelja. Na slučajan način izabere bit i promeni mu vrednost. Definiše se verovatnoća sa kojom se bit u hromozomu invertuje (verovatnoća mutacije).

### Permutaciono kodiranje

Koristi se kod problema uređivanja. Svaki hromozom je sekvenca elemenata čije se optimalno uređenje traži. Pri rekombinovanju i mutiranju se vodi računa o očuvanju konzistencije hromozoma. Pr. problem trgovačkog putnika (naći redosled gradova tako da se pređe najkraći put).

#### rekombinacija

Sa jednom tačkom prelazka. Kopira se sekvenca iz prvog roditelja to te tačke. Dodaju se one stavke iz drugog koje se već ne nalaze u potomku.

#### mutacija

Vrši se zamena određenih slučajno izabranih stavki iz hromozoma. Verovatnoća mutacije određuje koliko će zamena biti.

### Kodiranje vrednostima

Koristi se kod problem gde se koriste složenije vrednosti. Hromozom je sekvenca vrednosti određenog tipa. Primeri: 1,25 -5,3 ili AHGF ili nazad napred levo levo. Zahteva specijalizovane operacije rekombinacije i mutacije. Pr. nalaženje težina za neuronsku mrežu.

#### rekombinacija

Mogu se iskoristiti sve varijantne rekombinacije kao kod binarnog kodiranja.

#### mutacija

Dodaje se proizvoljna mala vrednost pojedinim vrednostima iz hromozoma. Verovatnoća mutacije definiše koliko vrednosti iz hromozoma će biti modifikovano i kolika će veličina modifikacije biti.

### Kodiranje stablom

Uglavnom se koristi za razvoj programa ili izraza (gentsko programiranje). Svaki hromozom predstavlja stablo koje u čvorovima čuva određene objekte: funkcije, operatore, konstante, promenljive i sl. Pr. nalaženje funkcije koja će da aproksimira zadati skup uređenih parova.

#### rekombinacija

Bira se po jedna tačka prelazka u stablima roditelja. Stablo potomka se dobija spajanjem jednog dela jednog i drugog dela drugog roditelja.

#### mutacija

Vrši se izmenama u proizvoljnim čvorovima. Mogu se menjati operatori, funkcije, promenljive i dr.

# Izbor roditelja

---

## Rulet sekcija

Što je veća dobrota jedinke veća je verovatnoća da bude izabrana za roditelja.

## Rangiranje

Rešava problem kada postoje velike razlike u dobrotama jedinki. Sortiraju se po dobroti pa im se dodele rangovi od 1 do N (veličina populacije). Na ovaj način sve jedinke imaju šanse da budu izabrane.

## Metod stablinog stanja

Novu generaciju treba da u velikoj meri čine najbolje jedinke iz prethodne generacije. Iz tekuće se izbacuje određeni broj najlošijih. Bira se par najboljih koje daju onoliko nvoih potomaka koliko je izbačeno. Ostatak populacije se prenosi u novu populaciju.

## Etilizam

Uveden je da bi se sprečilo gubljenje najboljih rešenja pri kreiranju nove populacije. Prvo se kopira par najboljih rešenja u novu populaciju, dok se ostatak nove generacije dobija rekombinacijom i mutacijom.

# Veštačke neuronske mreže

Oblast potiče od želje da se proizvedu sistemi za inteligentne proračune slične onima koje rutinski obavlja ljudski mozak. ANN uče na primerima (kao deca). Kod ANN nema eksplicitnog opisa problema, nema potrebe za programerom. Program sam sebe adaptira tokom treninga, na osnovu primera sličnih problema čak i bez postojanja željenog rešenja za svaki problem. Nakon treninga program je sposoban da poveže podatke o problemu sa rešenjem, ulaze sa izlazima, kao i da ponudi rešelje za potpuno novi problem.

ANN su posebno korisne za klasifikaciju (dijagnostika, prepoznavanje uzoraka, govora) i probleme aproksimacije funkcija. Predikciju vremenskih serija (finansijska) i predprocesiranje (klasterizacija, vizuelizacija podataka). Ne mogu se primeniti za probleme koje se odnose na manipulaciju simbolima.

Veštačka neuronska mreža ima osobinu da prepozna obrasce u zadatoj kolekciji podataka i da izgrađuje model za njih.

## Model veštačkog neurona (perceptron)

Veštački neuron predstavlja jednostavni procesni element koji obavlja relativno jednostavnu matematičku funkciju. Ulazi, težine, obrada, funkcija aktivacije, izlaz.

Pr. ulazi  $x$  i  $y$ , težine  $A$  i  $B$ , zbir na ulazu je  $Ax + By$ , prag  $C \Rightarrow$  izlaz je 1 ako je zbir veći a nula ako je manji. Dobiju se dva regiona u  $xy$  ravni podeljeni linijom  $Ax+By+C=0$ . Ako razmatramo ulaz kao tačku u ravni tada perceptron kaže u kom se regionu nalazi ta tačka.

Neural computing zahteva određeni broj perceptrona povezanih u neuronsku mrežu. Neuroni su organizovani u nivoe. Svaki neuron je jednostavna procesna jedinica koja dobija jedan ili više ulaza i proukuje izlaznu vrednost. Neuron jednostavno računa zbir ulaza i zračunava izlaz na osnovu praga i aktivacione funkcije. Sam proces učenja se svodi na podešavanje vrednosti težina.

*Kako ANN radi?* Postavi inicijalne vrednosti težina slučajno (male vrednosti  $[-1,1]$ ). Čita ulaz, računa izlaz i poredi ga sa očekivanim. Modifikuje težine u skladu sa rezultatom. Radi to određeni broj iteracija ili dok se ne dođe do određenog praga greške.

*Problemi:* funkcija aktivacije (nelin), estimacija greške (kvadratna), podešavanje težina ( $\downarrow$ ), broj neurona (više znači veću tačnost, sporije i over-fitting (memorisanje umesto razumevanja), reprezentacija podataka (pre-processing), veličina trening skupa (nema pravila, primeri moraju da reprezentuju generalnu populaciju, da sadrže članove svih klasa, širok skup varijacija)

Nakon svake iteracije podešavaju se težine da bi se minimizovala greška. Mogu se podesiti sve moguće težine ili back propagation (primer nadgledanog učenja). Znamo šta treba dobiti, znamo grešku u poslednjem nivou, i vraćamo se unazad. Greška na svakom skrivenom sloju je srednja vrednost procenjene greške.

## Arhitekture neuronskih mreža

Po načinu povezivanja:

1. sa povratim vezama – potpuno povezane i ćelijske
2. bez povratnih veta – slojevita neurnoska mreža

Slojevite neuronske mreže

Na ulaz jednog neurona iz sloja se dovode izlazi svih neurona iz prethodnog sloja, a njegov izlaz na sve neurone narednog sloja. Neuroni u prvom sloju imaju samo po jedan ulaz – ulazi u mrežu. Izlazi iz poslednjeg su izlazi iz mreže. Ostali su skriveni.

Da li su poznati očekivani izlazi – nadgledano i nenadgledano

Da li se pri obučavanju menja arhitektura (broj neurona, slojeva) – statičko i dinamičko